中山大學
SUN YAT-SEN UNIVERSITY



结构体进阶与练习

中山大学计算机学院

讲课人：万海

目录

CONTENTS

01 计算距离和

02 计算时间差

03 停车系统

04 银行系统

05

例题1-计算距离和

计算两个距离（英尺，英寸）的和。

示例1：输入第一个距离：23英尺8.6英寸；

输入第二个距离：33英尺2.4英寸。

输出：57英尺11英寸

例题1-计算距离和

```
struct Distance {
   int feet;
   float inch;
} d1, d2, result;
```

定义距离

```
// adding distances
result.feet = d1.feet + d2.feet;
result.inch = d1.inch + d2.inch;

// convert inches to feet if greater than 12
while (result.inch >= 12.0) {
   result.inch = result.inch - 12.0;
   ++result.feet;
}
printf("\nSum of distances = %d\'-%.1f\"", result.feet, result.inch);
return 0;
```

分别计算英尺和英寸部分

检查是否进位

例题2-计算时间差

计算两个时间（时分秒格式）的差（开始时间 - 结束时间）。

示例1 ： 输入开始的时分秒：13时34分55秒;
输入结束的时分秒：8时12分15秒。

输出： （13:34:55 - 8:12:15 = ）5:22:40

例题2-计算时间差

```c
#include <stdio.h>
struct TIME {
    int seconds;
    int minutes;
    int hours;
};

void differenceBetweenTimePeriod(struct TIME t1,
                                 struct TIME t2,
                                 struct TIME *diff);
```

定义时间结构体

例题2-计算时间差

```
// Computes difference between time periods
void differenceBetweenTimePeriod(struct TIME start,
                                 struct TIME stop,
                                 struct TIME *diff) {
   while (stop.seconds > start.seconds) {
      --start.minutes;
      start.seconds += 60;
   }
   diff->seconds = start.seconds - stop.seconds;
   while (stop.minutes > start.minutes) {
      --start.hours;
      start.minutes += 60;
   }
   diff->minutes = start.minutes - stop.minutes;
   diff->hours = start.hours - stop.hours;
}
```

如果秒部分不够减，进行借位

如果分钟部分不够减，进行借位

课堂练习 - 计算平面上多边形的周长

平面上有若干点构成凸多边形，顶点数 $N <= 50$，其顶点按顺序分别是 $\{p_0, p_1, \ldots, p_{n-1}\}$，第$i$点坐标是$(x_i, y_i)$。请编写一个程序完成多边形数据录入（最后一个点坐标等于第一个点坐标，坐标精度要求 0.001），周长计算，打印每个边的长度，以及周长。

练习要求：

- 定义描述问题需要的数据类型
- 申明问题求解需要的函数

例题3-停车系统

请你给一个停车场设计一个停车系统。停车场总共有三种不同大小的车位：大，中和小，每种尺寸分别有固定数目的车位。

请你实现 `ParkingSystem` 类：

- `ParkingSystem(int big, int medium, int small)` 初始化 `ParkingSystem` 类，三个参数分别对应每种停车位的数目。
- `bool addCar(int carType)` 检查是否有 `carType` 对应的停车位。 `carType` 有三种类型：大，中，小，分别用数字 `1`， `2` 和 `3` 表示。**一辆车只能停在** `carType` 对应尺寸的停车位中。如果没有空车位，请返回 `false` ，否则将该车停入车位并返回 `true` 。

示例 1：

输入：
`["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]`
`[[1, 1, 0], [1], [2], [3], [1]]`
输出：
`[null, true, true, false, false]`

例题3-停车系统

```
typedef struct {
    int b, m, s;
} ParkingSystem;
```

定义停车系统

```
ParkingSystem* parkingSystemCreate(int big, int medium, int small) {
    ParkingSystem* ret = malloc(sizeof(ParkingSystem));
    ret->b = big;
    ret->m = medium;
    ret->s = small;
    return ret;
}
```

构造一个停车系统

例题3-停车系统

```
bool parkingSystemAddCar(ParkingSystem* obj, int carType) {
    if (carType == 1) {
        if (obj->b > 0) {
            obj->b--;
            return true;
        }
    } else if (carType == 2) {
        if (obj->m > 0) {
            obj->m--;
            return true;
        }
    } else if (carType == 3) {
        if (obj->s > 0) {
            obj->s--;
            return true;
        }
    }
    return false;
}

void parkingSystemFree(ParkingSystem* obj) {
    free(obj);
}
```

减少对应停车位数量

例题4-银行系统

你的任务是为一个很受欢迎的银行设计一款程序，以自动化执行所有传入的交易（转账，存款和取款）。银行共有 `n` 个账户，编号从 `1` 到 `n` 。每个账号的初始余额存储在一个下标从 **0** 开始的整数数组 `balance` 中，其中第 `(i + 1)` 个账户的初始余额是 `balance[i]` 。

请你执行所有 **有效的** 交易。如果满足下面全部条件，则交易 **有效**：

- 指定的账户数量在 `1` 和 `n` 之间，且
- 取款或者转账需要的钱的总数 **小于或者等于** 账户余额。

实现 `Bank` 类：

- `Bank(long[] balance)` 使用下标从 **0** 开始的整数数组 `balance` 初始化该对象。
- `boolean transfer(int account1, int account2, long money)` 从编号为 `account1` 的账户向编号为 `account2` 的账户转帐 `money` 美元。如果交易成功，返回 `true` ，否则，返回 `false` 。
- `boolean deposit(int account, long money)` 向编号为 `account` 的账户存款 `money` 美元。如果交易成功，返回 `true` ；否则，返回 `false` 。
- `boolean withdraw(int account, long money)` 从编号为 `account` 的账户取款 `money` 美元。如果交易成功，返回 `true` ；否则，返回 `false` 。

示例：

输入：
```
["Bank", "withdraw", "transfer", "deposit", "transfer", "withdraw"]
[[[10, 100, 20, 50, 30]], [3, 10], [5, 1, 20], [5, 20], [3, 4, 15], [10, 50]]
```
输出：
```
[null, true, true, true, false, false]
```

例题4-银行系统

```
typedef struct {
    long long * balance;
    int balanceSize;
} Bank;

Bank* bankCreate(long long* balance, int balanceSize) {
    Bank * obj = (Bank *)malloc(sizeof(Bank));
    obj->balance = (long long *)malloc(sizeof(long long) * balanceSize);
    obj->balanceSize = balanceSize;
    memcpy(obj->balance, balance, sizeof(long long) * balanceSize);
    return obj;
}
```

定义和构造银行系统

例题4-银行系统

```
bool bankTransfer(Bank* obj, int account1, int account2, long long money) {
    if (account1 > obj->balanceSize || account2 > obj->balanceSize || obj->balance[account1 - 1] < money) {
        return false;
    }
    obj->balance[account1 - 1] -= money;
    obj->balance[account2 - 1] += money;
    return true;
}
```

涉及多种动作：两账户交易

```
bool bankDeposit(Bank* obj, int account, long long money) {
    if (account > obj->balanceSize) {
        return false;
    }
    obj->balance[account - 1] += money;
    return true;
}
```

存款

```
bool bankWithdraw(Bank* obj, int account, long long money) {
    if (account > obj->balanceSize || obj->balance[account - 1] < money) {
        return false;
    }
    obj->balance[account - 1] -= money;
    return true;
}
```

取款

```
void bankFree(Bank* obj) {
    free(obj->balance);
}
```

例题4-银行系统 - 扩展练习

应客户需求，需要开发一个对账功能。该功能输入一个客户号，要求按存取款的顺序打印交易过程。例如：

```
Account 7:
deposit     withdraw        balance
                            10.00
8.00                        18.00
            12.00           6.00
```

练习要求：

- 定义描述问题需要的数据类型
- 申明问题求解需要的函数

中山大學
SUN YAT-SEN UNIVERSITY



谢 谢

中山大学计算机学院

编制人：课题组